

Lab 0

1、a. 有的，实验前小组成员先对 `main.cpp` 的功能模块进行划分，约定各自负责的代码段落和修改范围，避免同时修改同一代码块。

b. 一方面可以将部分修改的代码挑选出来提交，减少与无关代码混在一起保存带来的不便；另一方面提供暂存区，可以帮助校验内容，减少手动合并出错的概率。总体来说，可以提升效率和修改的严谨性。

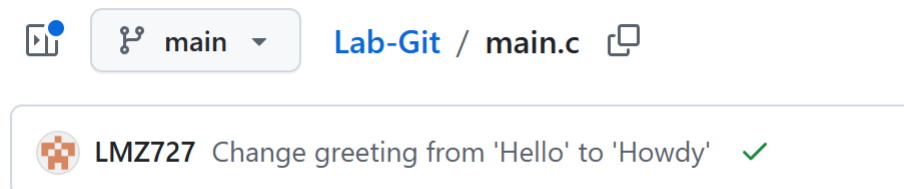
c. `git branch` 仅展示本地分支，而 `git branch -a` 能同时展示本地与远程跟踪分支。

3. 选择 a. Commit Message 规范：主要介绍了 Commit message 的 Angular 规范，分别阐释了 Commit message 的作用、格式、撰写工具（Commitizen）、检查工具（validate-commit-msg）和生成 Change log；

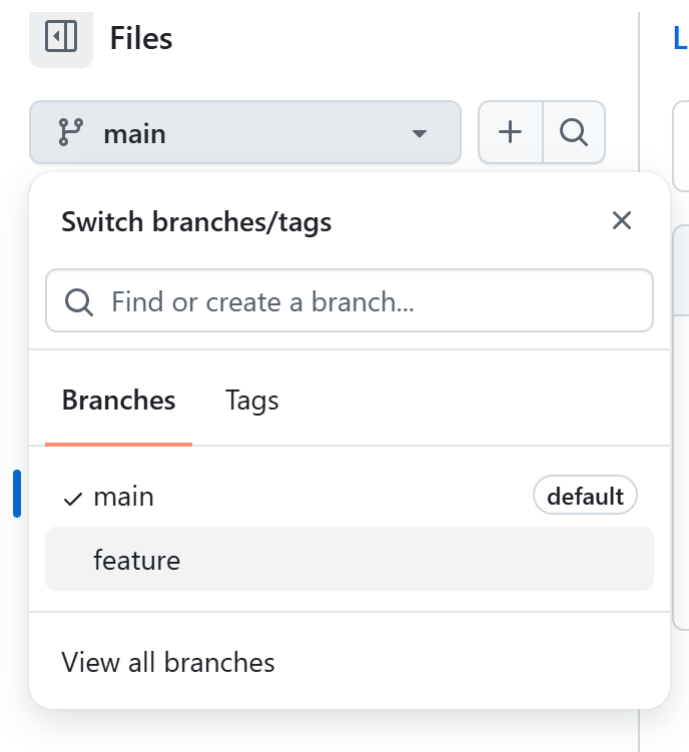
b. Git Flow 分支控制：主要介绍了 Gitflow 的分支管理规范，分别讲述了 Gitflow 的一些关键分支以及流程图和 Gitflow 的一些使用示例；

4. 实验步骤

(1) 使用模板仓库建立个人仓库，完成 `main.c` 文件中的 TODO 部分并进行一次了 commit；



(2) 新建 feature 分支，修改 `main.c` 并提交



(3) 切回 `main` 分支，修改同一处并提交

```

1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      <<<<<< feature (Current change)
7      printf("Hi, ICS!\n");
8      =====
9      printf("Howdy, ICS!\n");
10     >>>>>> main (Incoming change)
11 }
12

```

(4) 发起 Merge, 解决合并冲突

1 conflicting file	main.c	✓ Resolved
main.c main.c	<pre> 1 #include <stdio.h> 2 3 int main() 4 { 5 // @TODO: print a sentence you want. 6 printf("Hi, ICS!\n"); 7 } </pre>	